



សាកលវិទ្យាល័យភ្នំពេញអន្តរជាតិ
PHNOM PENH INTERNATIONAL UNIVERSITY

URBANCART E-COMMERCE REPORT

PROJECT REPORT

BIG DATA

Prepared by:

Group 8

Ly Seavmey

Ath Molita

Chamrouen Vichita

Kheng Solida

Submitted to: Mr. Min Sothearith

May Term, 2026

Faculty of Science and Information Technology

Phnom Penh International University

Table of Contents

1. Executive Summary	1
2. Introduction & Business Context	2
3. Data Description	3
4. SQL Extraction & Methodology	7
5. Data Cleaning & Integration	12
6. Numpy Analytical Methods	24
7. Findings & Insights	30
8. Limitations & Threats to Validity	36
9. Recommendations & Next	36
10. Appendices	37

1. Executive Summary

This project analyzes UrbanCart, a mid-size online retail dataset, to understand customer behavior, product performance, demand, and data quality. The available data includes 2,500 customers, 300 products, 9,000 orders, 20,362 order-item records, 4,000 reviews, and 12,000 web sessions. In addition to the main database, we also worked with a legacy customer export and a 2024 product catalog. Before using these sources for analysis, we had to clean and reconcile several data-quality issues.

Our analysis shows that Books & Media generated the highest revenue, with \$2,288,541.26, followed by Beauty & Health with \$1,781,007.78. However, the raw profitability results were affected by three unusually high product prices. These products were identified using the IQR method, with an upper threshold of \$1,160.36. After capping the extreme prices for a sensitivity analysis, Electronics became the highest-margin category at 52.44%. This shows why checking unusual values is important before making business decisions from the data.

For customer analysis, our RFM segmentation identified the Champions group as the most valuable segment. This segment contains 340 customers and generates \$2,787,568.97 in revenue. We also identified a potential win-back group of 1,238 high-spend customers who have not purchased for more than six months. This group could be useful for targeted retention campaigns.

We also investigated whether customer review ratings are related to repeat purchasing. The result showed only a very weak relationship, with a correlation of 0.021. Repeat customers had an average rating of 3.96, compared with 3.92 for customers who purchased only once. Based on these results, review rating alone does not appear to be a strong indicator of whether a customer will purchase again.

For revenue forecasting, we implemented linear regression using the NumPy normal equation. The model explains approximately 67.0% of the variation in historical monthly revenue. It predicts approximately \$346,488.59 for the next month and \$353,754.30 for the following month. Since the model only uses a linear time trend, we consider these values to be planning estimates rather than exact predictions.

Finally, the project showed that data quality has a direct effect on business conclusions. Problems such as inconsistent customer dates, duplicate records, missing values, invalid ratings, and extreme prices can change the results of the analysis. Therefore, our main recommendation is to keep the cleaning and validation process as part of the regular analytical workflow instead of treating it as a one-time task.

2. Introduction & Business Context

UrbanCart is a fictional mid-size online retailer that operates across six product categories and fifteen cities. For this project, our goal is to use the available business data to understand customer behavior, product performance, and possible business risks. At the same time, we need to make sure that the data is reliable enough to support our conclusions.

The project requires us to use SQL, pandas, NumPy, and data visualization as part of one complete analytical workflow. We therefore divided the work into several stages. First, we used SQL to extract and summarize information from the database. Next, we cleaned and combined the database with the external customer and product files using pandas. After that, we used NumPy to implement the required analytical methods, including RFM segmentation, cosine similarity, linear regression, and Monte Carlo simulation. Finally, we used charts and summary tables to explain the results and connect them to business decisions.

2.1. Customer Value and Loyalty

The first business question focuses on customers. We want to know which customers are the most valuable, what characteristics or behaviors are associated with valuable customers, and which high-value customers may be at risk of becoming inactive.

To answer these questions, we use several approaches. We calculate customer lifetime spending, examine repeat-purchase behavior, and create RFM segments based on Recency, Frequency, and Monetary value. We also look at customer engagement and identify high-spending customers who have not purchased recently. These results can help UrbanCart decide which customers should receive retention or win-back campaigns.

2.2. Product Economics and Demand

The second business question focuses on products and sales. We want to understand which categories generate the most revenue, which products have high or low return rates, and whether unusual prices affect the profitability results.

We also examine product demand and inventory risk. The analysis includes product rankings, monthly revenue trends, a simple revenue forecast, and a Monte Carlo simulation for stockout risk. These methods allow us to move beyond simply asking which products sell the most and instead consider which products may require attention from the business.

2.3. Data Quality and Trustworthiness

The third business question is about whether the data can be trusted. During the project, we found that the external files were not immediately ready for analysis. The legacy customer export contains inconsistent date formats, duplicate or near-duplicate customer records, missing emails, and other issues. The 2024 product catalog also uses different column names and only partially overlaps with the product information in the main database.

Because of these issues, data cleaning is an important part of our analysis. We do not want to produce business recommendations based on incorrect or inconsistent records. Instead, we document the cleaning decisions and keep the original meaning of the data whenever possible.

2.4. Analytical Policy

To keep the analysis consistent, we follow several rules throughout the project. Completed orders are treated as realized sales, while cancelled and pending orders are excluded from realized revenue. Negative quantities in order items are treated as returns and therefore reduce net revenue. Invalid review ratings are excluded from rating-based calculations.

For missing customer information such as age, city, and gender, we do not automatically replace missing values with zero or an arbitrary category. Instead, the missing values are kept and flagged. This allows us to distinguish between information that is actually known and information that is unavailable.

Overall, the purpose of this project is not only to produce charts and calculations. Our goal is to build a complete and reproducible workflow that starts with raw business data, identifies and handles data-quality problems, performs the required analysis, and ends with practical recommendations for UrbanCart.

3. Data Description

This section describes the structure, sources, size, and limitations of the data used in our UrbanCart analysis. Understanding these points is important before performing the SQL queries, data cleaning, and statistical analysis in the later sections.

The main database contains information about customers, products, orders, order items, reviews, and web sessions. The two external files provide additional customer and product information, but they do not follow exactly the same structure as the database. Because of this, we needed to standardize some fields and identify suitable keys before combining the datasets.

3.1. Schema / ERD

The main UrbanCart database is stored in ecommerce.db and contains six main tables:

Table	Purpose
customers	Contains customer information such as name, email, signup date, city, country, age, and gender.
Products	Contains product information including product name, category, price, cost, stock, and rating.
Orders	Contains order-level information such as order date, customer, payment method, and order status.

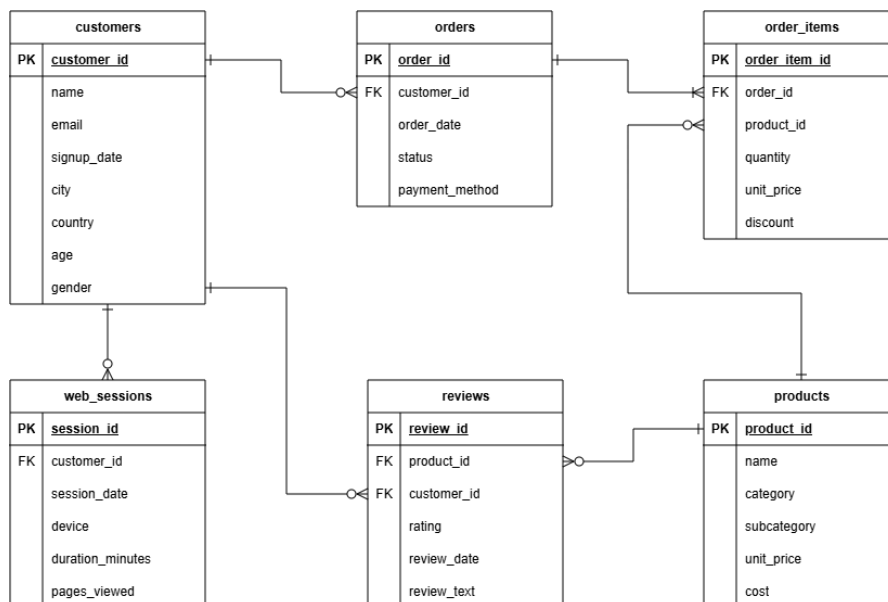
Order_items	Contains the products and quantities included in each order, together with prices and discounts.
Reviews	Contains customer ratings and review text for products.
Web_sessions	Contains information about customer website sessions, including device duration, pages viewed and country.

The main relationships between the tables allow us to connect customers, orders, and products. The most important relationship is: **Customers** → **Orders** → **Order Items** → **Products**

A customer can have multiple orders, and each order can contain multiple order items. Each order item refers to a particular product. This structure allows us to calculate customer spending, product sales, category revenue, and other business metrics.

The reviews table provides additional information about customer feedback and product ratings, while web_sessions provides information about website engagement such as device type, session duration, and pages viewed.

ERD



The ERD should show the six database tables, their primary keys, foreign keys, and relationships between them.

3.2. Data Sources

1. Main UrbanCart Database

The primary source is ecommerce.db. This database contains the operational data used for most of the SQL analysis, including customers, products, orders, order items, reviews, and web sessions.

We use this database as the main source for calculating revenue, customer spending, product performance, return rates, customer reviews, and website engagement.

2. Legacy Customer Export

The first external dataset is the legacy customer export, **legacy_customers_export.csv**.

This file contains historical customer information that is useful for checking whether customer records in the current database are complete and consistent. However, the file is not immediately ready to use because it contains several data-quality problems.

One of the main issues is that customer dates appear in four different formats. Customer names and other identity fields also have inconsistent capitalization and formatting. There are also duplicate or near-duplicate customer records, missing email addresses, and some records that do not contain useful customer information.

Because of these issues, we do not directly append the legacy file to the current customer table. Instead, we first standardize the fields, normalize customer identities, investigate possible duplicates, and then match records to the current customer data where there is enough evidence to do so.

3. 2024 Product Catalog

The second external dataset is **product_catalog_2024.csv**.

This file provides another source of product information from 2024. It contains product identifiers and product attributes that can be compared with the products table in the main database.

The main challenge is that the column names and structure are not exactly the same as the database table. Therefore, we standardize the field names before comparing the two sources.

We use the SKU/product ID as the main integration key. The comparison shows that some products appear in both sources, while others appear only in the catalog or only in the database. These differences are important because they may represent new products, discontinued products, or differences between the two source systems.

3.3.Data Scale

The main database contains the following records:

Table	Number of Records
Customers	2500
Products	300
Orders	9000
Order Items	20362
Reviews	4000
Web Sessions	12000

The dataset is large enough to perform customer-level, product-level, and time-based analysis.

For example, the 9,000 orders and 20,362 order-item records provide enough transaction data to compare product categories and analyze monthly revenue. The 2,500 customer records allow us to segment customers using RFM analysis and identify high-value or potentially inactive customers.

The 4,000 reviews allow us to examine product ratings and investigate whether customer satisfaction is related to repeat purchasing. The 12,000 web sessions also provide information about customer engagement across different devices and countries.

In addition to the database, the project includes the two external CSV files. These files are important for testing our ability to clean and integrate data from different sources rather than relying only on an already-structured database.

3.4. Known Limitations

Although the dataset provides enough information for the required analysis, there are several limitations that we need to consider when interpreting our results.

- **Data-quality limitations**

The legacy customer export contains inconsistent date formats, duplicate or near-duplicate customer records, missing values, and inconsistent formatting. These issues require additional cleaning before the data can be safely integrated with the current customer records. The product catalog also does not completely match the product table in the main database. Some products appear in one source but not the other, so the two sources cannot simply be appended together.

- **Missing values**

Some customer fields, such as age, city, and gender, contain missing values. Review text is also not available for every review record. We decided not to automatically

replace these missing values with zero because zero would imply that the value is actually known. Instead, missing values are kept or flagged depending on the analysis.

- **Extreme product prices**

The product data contains several unusually high prices. These values affect revenue and margin calculations significantly. We therefore use an IQR-based method to identify price outliers and perform a sensitivity analysis. However, we do not assume automatically that an outlier is an error. A high price could represent a genuine product, so the original values are retained while the capped values are used for comparison.

- **Limited forecasting information**

The revenue forecasting model uses historical monthly revenue and a relatively simple linear trend. It does not include external factors such as advertising campaigns, holidays, competitor activity, economic conditions, or changes in customer behavior. Therefore, the forecast should be treated as a planning estimate rather than an exact prediction.

- **Web-session limitation**

The web-session data provides information such as device type, duration, and pages viewed, but it does not provide a complete time-ordered customer journey from website visit to purchase. Because of this, the device-country conversion analysis should be interpreted as a conversion proxy, not as a complete marketing funnel measurement.

- **Inventory simulation limitation**

The Monte Carlo stockout analysis uses historical demand to simulate future demand. However, supplier lead times and replenishment schedules are not available in the dataset. Therefore, the stockout probabilities should be treated as risk indicators rather than exact predictions of future stockouts.

4. SQL Extraction & Methodology

The Extraction Process Step-by-Step

1. Database Connection: Python opens a direct bridge to **ecommerce.db** using **sqlite3.connect()**.
2. Direct Loading: **pd.read_sql_query()** executes the query.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('../data/raw/ecommerce.db')
df_category_perf = pd.read_sql_query(q1_sql, conn)
```

- **SQL Data Extraction**

Query 1: Total revenue, order count, and AOV by category (net of discounts)

```

SELECT
    p.category,
    COUNT(DISTINCT oi.order_id) AS total_orders,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS
net_total_revenue,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) /
COUNT(DISTINCT oi.order_id) AS avg_order_value
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status != 'Cancelled'
GROUP BY p.category
ORDER BY net_total_revenue DESC;

```

category	total_orders	net_total_revenue	avg_order_value
Books & Media	2927	3675270.95200003	1255.64432934747
Beauty & Health	2821	2957229.7545	1048.29129900745
Apparel	2854	1714435.75699999	600.713299579533
Sports & Outdoors	2820	1664660.6315	590.305188475178
Electronics	2891	1653446.69999999	571.928986509855
Home & Kitchen	2927	1414051.565	483.106103518961

- Books & Media generates the highest revenue and highest average order value, making it the strongest category financially.

Query 2 : Top 20 customers by lifetime spend including city and signup date

```

SELECT
    c.customer_id,
    c.name,
    c.city,
    c.signup_date,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS
lifetime_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status != 'Cancelled'

```

```
GROUP BY c.customer_id, c.name, c.city, c.signup_date
ORDER BY lifetime_spend DESC
LIMIT 20;
```

customer_id	city	signup_date	lifetime_spend
611	Sydney	2021-10-15	146989.767
689	Toronto	2020-08-25	108053.554
2181	Melbourne	2019-11-08	106449.5375
1259	Los Angeles	2023-02-25	95535.575
1773	Los Angeles	2022-02-13	80201.6235000001
1217	Paris	2022-07-07	79992.056
528	Vancouver	2023-10-12	73939.513
2445	Bangkok	2023-11-26	72592.959
1347	Paris	2020-05-28	71844.5975
1993	Sydney	2020-04-27	71364.757

- Customer 611 is the most valuable customer in the results, based on lifetime spending.

Query 3: Top 10 products with highest rating having at least 15 reviews

```
SELECT
    p.product_id,
    p.name AS product_name,
    p.category,
    ROUND(AVG(r.rating), 2) AS avg_rating,
    COUNT(r.review_id) AS review_count
FROM products p
JOIN reviews r ON p.product_id = r.product_id
GROUP BY p.product_id, p.name, p.category
HAVING COUNT(r.review_id) >= 15
ORDER BY avg_rating DESC, review_count DESC
LIMIT 10;
```

product_id	product_name	category	avg_rating	review_count
200	White Fitnes	Sports & Outdoors	4.6	15
19	Improve Smartphone	Electronics	4.53	15
191	Yeah Camping	Sports & Outdoors	4.53	15
283	Strategy Children	Books & Media	4.53	15
21	Right Camera	Electronics	4.5	16
66	Wide Cookware	Home & Kitchen	4.5	16
193	Add Camping	Sports & Outdoors	4.47	15
208	Series Skincare	Beauty & Health	4.42	19
182	Answer Fitnes	Sports & Outdoors	4.4	15
122	Attorney Men	Apparel	4.35	17

- These products have strong customer ratings with enough reviews to provide some evidence of customer satisfaction. They could be good candidates for promotion or recommendations.

Query 4: Web Engagement by Device for Purchasing Customers

```

SELECT
    ws.device,
    ROUND(AVG(ws.duration_minutes), 2) AS avg_session_duration_mins,
    ROUND(AVG(ws.pages_viewed), 2) AS avg_pages_viewed
FROM web_sessions ws
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = ws.customer_id
        AND o.status != 'Cancelled'
)
GROUP BY ws.device;

```

device	avg_session_duration_mins	avg_pages_viewed
desktop	6.25	4.7
mobile	6.23	4.64
tablet	6.11	4.74

- **Desktop** users spend the longest time on the site, while tablet users view the most pages. However, the differences are quite small, meaning engagement is relatively similar across devices.

Query 5: Return Rate by Product Category

```
WITH category_return_stats AS (
    SELECT
        p.category,
        COUNT(CASE WHEN oi.quantity < 0 THEN 1 END) AS
returned_items_count,
        COUNT(oi.order_item_id) AS total_items_count
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    GROUP BY p.category
)
SELECT
    category,
    returned_items_count,
    total_items_count,
    ROUND((1.0 * returned_items_count / total_items_count) * 100, 2) AS
return_rate_pct
FROM category_return_stats
ORDER BY return_rate_pct DESC;
```

category	returned_items_count	total_items_count	return_rate_pct
Beauty & Health	106	3354	3.16
Electronics	106	3401	3.12
Apparel	102	3372	3.02
Books & Media	93	3455	2.69
Home & Kitchen	91	3468	2.62
Sports & Outdoors	85	3312	2.57

- Beauty & Health has the highest return rate (3.16%), while Sports & Outdoors has the lowest (2.57%).

5. Data Quality & Integration

5.1. Overview of Data Cleaning Process

Before performing the main analysis, we checked the UrbanCart database and the two external CSV files for problems that could affect our results. The project data was intentionally designed to contain realistic data-quality issues, so we treated the raw data as something that needed to be investigated rather than assuming it was already clean.

Our cleaning process focused on the following areas:

1. Inconsistent legacy customer dates
2. Duplicate and near-duplicate customer records
3. Missing customer and review information
4. Invalid review ratings
5. Extreme product prices
6. Differences between the product catalog and database
7. Exact duplicate order-item records
8. Negative quantities representing returns
9. Reshaping data for analysis
10. Creating a consistent datetime index for time-series analysis

For each issue, we applied a specific rule instead of making changes automatically. We also compared the data before and after cleaning whenever possible.

The overall process was:

Raw Data → Quality Check → Cleaning Rule → Before/After Comparison → Integrated Data → Analysis

5.2. Standardizing Legacy Customer Dates

Problem

The legacy customer export contains **four different date formats** across its rows. The assignment specifically requires these formats to be standardized into a single datetime type. Having different date formats in the same column can cause problems when sorting customers, calculating customer age or tenure, or performing time-based analysis.

Cleaning rule

We converted the different date representations into a single pandas datetime64 format. We also checked the converted values to make sure that the dates were interpreted consistently. We did **not** replace invalid or missing dates with arbitrary values because doing so could create false signup dates.

Before and after evidence

Check	Before cleaning	After cleaning
Data type	String	datetime64[us]
Date formats	Multiple formats	Single datetime format
Example 1	30-Nov-2023	2023-11-30
Example 2	August 19, 2020	2020-08-19
Example 3	07/05/2023	2023-07-05
Example 4	2019-09-25	2019-09-25
Parsed dates	372	1426
Missing/unparseable dates	1055	1 (missing)

The notebook output shows that 1,426 dates were successfully parsed, while 1 value became missing because it could not be reliably parsed. The resulting column has a datetime dtype.

Result

After conversion, the legacy signup dates were stored consistently as datetime values. For example, 30-Nov-2023 became 2023-11-30, while August 19, 2020 became 2020-08-19.

This standardization makes the legacy dates compatible with the current customer data and allows them to be used in date-based operations such as sorting, filtering, calculating customer age at signup, and time-based analysis.

What we chose not to fix

We did not guess the value of the one unparseable date. Instead, it was converted to a missing value using `errors="coerce"`. We considered this safer than manually assigning a date without evidence, because an incorrect signup date could affect later customer analysis.

5.3. Duplicate and Near-Duplicate Customer Records

The legacy customer data needed to be checked for duplicate and near-duplicate customer records before combining it with the current customer table. The main issue was that the same customer could appear with differences in email formatting, name casing, or other identifying fields. If these records were treated as separate customers, their transaction history could be split across multiple customer IDs.

1. Exact Customer Matching

We first normalized the email addresses by removing unnecessary spaces and converting the values to lowercase. This allowed emails such as uppercase/lowercase variations to be treated as the same identifier.

After email normalization, the 1,427 legacy customer records were compared with the current customer table. 1,369 records were matched using normalized email addresses, while 58 records remained unmatched.

Matching stage	Records	Result
Legacy customer records	1427	Starting dataset
Matched by normalized email	1369	Accepted
Unmatched after email matching	58	Sent to secondary matching

The email-based matching was used as the primary rule because email provides a stronger customer identifier than name alone.

2. Secondary Customer Matching

For the 58 records that could not be matched by email, we performed a secondary matching process using customer names and city information. Name similarity was calculated using string similarity, while city information was used as additional supporting evidence.

We did not automatically accept every fuzzy match. Instead, we used the following rules:

- An exact normalized name match with the same city was considered a high-confidence match.

- An exact normalized name match with missing city information could also be accepted when there was no conflicting information.
- A less certain name similarity was only accepted when there was strong supporting city evidence.
- Records with weak similarity or insufficient identifying information were left unresolved.

The process resulted in 56 high-confidence matches and 2 unresolved records.

Secondary matching result	Records
Unmatched records investigated	58
High-confidence matches	56
Unresolved records	2

3. Records We Chose Not to Merge

We deliberately did not force the remaining two records into the current customer table.

One record contained the name test account, while its best candidate was stacey hunt. The name similarity was only 0.6087, so there was not enough evidence to conclude that they represented the same person.

The second unresolved record had a missing legacy customer name, while the proposed current customer was michael cross dds. Since there was no reliable name evidence and no supporting city information from the legacy record, we did not merge the records.

This decision was made to prioritize avoiding false customer matches. Incorrectly merging two different customers could combine their transactions and affect later customer-level analysis such as spending, RFM scores, and customer segmentation.

Final Integration Result

Overall, the customer matching process reduced the uncertainty in the legacy data without forcing uncertain records into the current customer table:

1,427 legacy customers → 1,369 email matches → 58 secondary matches → 56 accepted + 2 unresolved

The 56 high-confidence secondary matches were accepted for integration, while the 2 unresolved records were retained as unresolved rather than being assigned to an existing customer.

5.4. Missing Values

We checked the customer and review datasets for missing values in the fields that could affect the later analysis. The fields specifically examined were age, city, gender, and review text. Instead

of replacing missing values with arbitrary values such as 0, we created separate missing-value flags so that the original missing information was preserved.

The notebook identified the following missing values:

Field	Missing records	Treatment
Age	139	Flagged
City	63	Flagged
Gender	115	Flagged
Review text	838	Flagged

Age

There were 139 customer records with missing age values. We chose not to replace these values with 0 or an estimated age, because doing so could create misleading customer information and affect age-based analysis. Instead, the records were retained and marked using an `age_missing` flag. This allows later analysis to distinguish between a customer whose age is genuinely known and one whose age was not provided.

City

There were 63 missing city values. We did not infer a city from another field because there was not enough evidence to reliably determine the customer's location. The missing values were therefore retained and identified using the `city_missing` flag. This also avoids introducing incorrect location information that could affect geographic analysis or customer matching.

Gender

There were 115 missing gender values. We chose not to impute gender because assigning a gender based on a customer's name or other information could introduce incorrect assumptions. Instead, the original missing values were retained and marked using the `gender_missing` flag.

Review Text

There were 838 reviews with missing review text. We did not replace missing review text with artificial text or a numeric value. Missing review text does not necessarily mean that the review record itself is invalid, so the review record was retained and identified using `review_text_missing`. This is particularly important for later review analysis because a missing review should not be treated as if the customer actually wrote a review containing a value such as 0.

Overall Missing-Value Policy

Our general policy was to flag rather than silently impute missing values when there was no reliable basis for determining the original value. This preserves the original information and makes the missingness visible during later analysis.

We specifically chose not to use `fillna(0)` across the dataset, because zero does not have the same meaning as an unknown value. For example, an unknown age is not the same as an age of zero, and missing review text is not the same as a review containing the value zero.

The notebook's output confirms 139 missing ages, 63 missing cities, 115 missing genders, and 838 missing review texts.

5.5. Invalid Review Rating

Problem

We checked the review dataset to identify rating values that were outside the expected 1–5 rating scale. Since the rating is used in later customer and review analysis, invalid values needed to be handled before calculating rating distributions or other review-based metrics.

Cleaning rule

We treated ratings from 1 to 5 as valid. Any rating outside this range was considered invalid. The invalid records were removed from the cleaned review dataset rather than being converted to another rating. We did not replace invalid ratings with the mean, median, or another value because there was no reliable basis for determining what the customer's original rating should have been.

Check	Before cleaning	Cleaning rule	After cleaning
Total review records	4000	Remove invalid ratings	3959
Valid ratings (1-5)	3959	Keep	3959
Invalid rating	41	Remove	0
Missing rating	0	No action	0

Therefore, 41 records (1.03% of the original review dataset) were removed because their ratings were outside the accepted 1–5 range.

Rating distribution

We also compared the rating distribution before and after cleaning.

Rating	Original distribution	Cleaned distribution
1	211	211
2	292	292
3	607	607
4	1240	1240
5	1609	1609
Total valid ratings	3959	3959

The distribution of valid ratings remained unchanged after cleaning. This indicates that the cleaning step only removed the invalid rating records and did not alter any valid 1–5 ratings.

What we chose not to fix

We chose not to convert the 41 invalid ratings into valid ratings. For example, we did not replace them with the average rating or round them to the nearest valid rating.

This was because the correct rating for those records could not be determined from the available data. Assigning a new rating would introduce information that was not present in the original dataset. Removing the invalid records was therefore considered safer for the accuracy of the review analysis.

Final decision

The cleaned review dataset contains 3,959 valid review records, with ratings restricted to the expected 1–5 scale. The valid rating distribution was preserved, while the 41 invalid records were excluded from further rating analysis.

5.6. Negative Quantities and Returns

We checked the quantity field in the order-item dataset to identify transactions with negative quantities. Rather than automatically treating negative quantities as data errors, we interpreted them as return transactions, because a negative quantity represents items being reversed from a sale.

Cleaning rule

We used the following classification rule:

- quantity > 0 → Sale
- quantity < 0 → Return
- quantity = 0 → Zero quantity

We also created an `is_return` indicator and a `transaction_type` field so that return transactions could be identified during later analysis.

Before and after evidence

The order-item dataset contained 20,362 records. Of these, 19,779 had positive quantities and 583 had negative quantities. The negative-quantity records represented 2.86% of all order-item records.

We also checked for zero-quantity records and found none.

Check	Before cleaning	Cleaning rule	After cleaning
Total order-item rows	20,362	Retain transactions	20,362
Positive quantities	19,779	Classify as Sale	19,779
Negative quantities	583	Classify as Return	583
Zero quantities	0	No action	0

The row count remained **20,362** because we did not remove the return transactions.

Revenue impact

We also checked how the returns affected revenue. Normal sales generated \$14,069,986.11, while negative-quantity transactions contributed -\$419,250.56. After accounting for the returns, the resulting net revenue was \$13,650,735.55.

Revenue measure	Amount
Revenue from normal sales	\$14,069,986.11
Revenue impact from returns	-\$419,250.56
Net revenue	\$13,650,735.55

This shows why the negative quantities should not simply be deleted. Removing the 583 return records would cause the calculated revenue to be overstated by \$419,250.56.

What we chose not to fix

This shows why the negative quantities should not simply be deleted. Removing the 583 return records would cause the calculated revenue to be overstated by \$419,250.56

We also did not find any zero-quantity records requiring additional treatment.

The main reason for retaining the returns was that they contain useful business information. Removing them would not only reduce the transaction history but would also prevent the analysis from reflecting the financial impact of returned products.

5.7. Product Price Outliers and Sensitivity Analysis

We checked the product price data for unusually high values using the Interquartile Range (IQR) method. This was done to identify prices that were statistically different from the majority of products and to determine whether these values could have a significant effect on the analysis.

Outlier detection rule

The first quartile (Q1) of product prices was \$192.33, while the third quartile (Q3) was \$579.54. This produced an IQR of \$387.21.

We calculated the upper outlier boundary using: **Upper bound = Q3 + 1.5 × IQR**

This resulted in an upper bound of approximately \$1,160.36. Any product with a price above this value was therefore flagged as a potential price outlier.

Outlier identified

Three products exceeded the upper bound:

product_id	Product price
245	\$11,506.50
250	\$8,284.00
270	\$34,872.00

These prices are substantially higher than the upper IQR boundary, so they were flagged for further investigation.

Sensitivity analysis

We did not immediately remove or replace the three products. Instead, we compared the product-price distribution using all 300 products with a second version excluding the three flagged products.

Metric	All products	Excluding flagged outliers
Product count	300	297

Minimum price	\$5.69	\$5.69
Mean price	\$576.95	\$398.73
Median price	\$380.76	\$376.48
Maximum price	\$34,872.00	\$978.72
Q1	\$192.33	\$192.19
Q3	\$579.54	\$567.69

The results show that the three extreme values have a large effect on the mean and maximum price. The mean decreased from \$576.95 to \$398.73 when the outliers were excluded, while the median changed only slightly from \$380.76 to \$376.48.

This suggests that the extreme prices strongly affect average product price but have much less effect on the typical product price represented by the median.

Checking the business impact

We also checked whether the three flagged products were actually used in transactions. Products 245, 250, and 270 appeared in 192 order-item rows in total.

Product ID	Order-item rows	Total quantity	Revenue
245	65	67	\$770,935.50
250	69	83	\$687,572.00
270	58	65	\$2,266,680.00
Total	192	215	\$3,725,187.50

Although these products only appeared in approximately 0.94% of the 20,362 order-item rows, they generated \$3,725,187.50, which represents 27.29% of total net revenue.

This was an important reason for not deleting the three products. Their prices are unusual statistically, but they also represent a substantial amount of actual business revenue.

What we chose not to fix

We chose not to cap, replace, or delete the three price outliers. The IQR method identifies statistically unusual values, but it does not prove that those values are incorrect.

In addition, the three products appeared in actual transactions and their transaction prices were consistent with their product prices. Therefore, we did not have sufficient evidence to classify them as data-entry errors.

Instead, we flagged the three products as potential price outliers and used an outlier-excluded version only for sensitivity analysis.

Final decision

The three products were retained in the main dataset and flagged as price outliers. The sensitivity analysis is used to show how much these products influence price statistics

Our decision was: Statistically unusual does not automatically mean incorrect. We retained the three products because removing them would exclude 27.29% of observed revenue and there was no sufficient evidence that their prices were erroneous.

This approach allows the analysis to remain faithful to the original business data while making the influence of extreme prices transparent.

5.8. Product Catalog Integration

The external product_catalog_2024.csv used different column names from the database, such as SKU instead of product_id and dept instead of category. We standardized these names and used the product identifier to compare the two sources. The project also required us to identify products that appeared in only one source.

Check	Result
Products in both sources	57
Database-only product	45
Catalog-only SKUs	12

Products found in both sources were matched and integrated. Products that appeared only in the external catalog were kept as supplier-only records rather than being added to the transactional database, since there was no evidence that they had corresponding sales records.

We chose **not to force unmatched SKUs to match** because doing so could create incorrect product records. The original database products were also retained even when they had no corresponding catalog entry.

Implementation: The column standardization and product integration were performed in the accompanying Jupyter notebook.

5.9. Exact Duplicate Order Items

We checked the `order_items` table for exact duplicate rows to make sure the same transaction was not recorded more than once. Exact duplicates could cause quantities and revenue to be counted multiple times.

Cleaning rule

We identified rows where all relevant order-item fields were identical. Exact duplicate rows were removed, while records that were only similar but had different transaction information were retained.

Before and after

Check	Result
Total order-item rows before cleaning	20,362
Exact duplicate rows	0
Total rows after cleaning	20,362

5.10. Data Reshaping

We reshaped the cleaned order data using a `pivot_table` to organize revenue by product category and month. This changed the data from transaction-level records into a summary format that makes it easier to compare revenue across categories over time.

This transformation did not remove or modify the original transaction records. It only created a summarized view for analysis.

Implementation: The reshaping was performed in the accompanying Jupyter notebook using `pandas pivot_table`.

5.11. Time-Series Analysis

We converted the relevant date field to a `datetime` type and used the cleaned transaction data to examine revenue over time. The data was grouped by month so that changes in revenue could be compared across different periods.

The time-series transformation was performed after the cleaning and integration steps, ensuring that invalid ratings, return handling, and other data-quality issues were already addressed.

The resulting monthly revenue data provides a consistent time-based view of sales performance and can be used for later trend analysis and visualization.

Implementation: The time-series analysis was performed in the accompanying Jupyter notebook using pandas datetime functionality and grouping/resampling.

6. Numerical Analysis with NumPy

6.1. RFM Customer Segmentation

We used RFM (Recency, Frequency, Monetary) analysis to evaluate customer purchasing behavior. The calculation was performed for 2,438 customers using the cleaned sales and order data.

The analysis date was set to the latest order date in the dataset: 2024-12-31 23:58:14

RFM calculation

For each customer, we calculated three measures:

- **Recency:** number of days since the customer's most recent purchase.

$$R_i = D_{analysis} - D_{lastPurchase,i}$$

-

Frequency: number of unique orders made by the customer.

$$F_i = \text{number of unique orders by customer } i$$

-

Monetary: total net revenue generated by the customer.

$$M_i = \sum \text{net revenue for customer } i$$

The calculations were implemented using NumPy arrays. Recency was calculated as the difference between the analysis date and each customer's last purchase, while frequency counted unique order IDs and monetary value was calculated by summing the customer's revenue.

RFM scoring

We divided the customers into five score groups using the 20th, 40th, 60th, and 80th percentiles.

For Frequency and Monetary:

- Higher values receive higher scores.
- Scores range from **1 to 5**.

For Recency, the scoring is reversed:

- More recent purchases receive higher scores.
- A customer with a lower number of days since their last purchase receives a higher Recency score.

The three scores were then combined using:

RFM Score = R Score + F Score + M Score or

Therefore, the final RFM score ranges from **3 to 15**, where a higher score represents stronger overall customer activity and value.

	customer_id	recency	frequency	monetary	R_score	F_score	M_score	RFM_score
0	1	372	3	2368.55	1	3	2	6
1	2	79	4	4325.62	4	4	3	11
2	3	279	6	12763.42	2	5	5	12
3	4	780	1	24.70	1	1	1	3
4	5	72	2	4238.67	4	2	3	9

For example, **Customer 3** has an RFM score of **12**, mainly because of a high frequency score of 5 and monetary score of 5. In contrast, **Customer 4** has an RFM score of only **3**, reflecting low frequency, low monetary value, and a long period since the last purchase.

Result

The RFM analysis produced a numerical customer profile for all **2,438 customers**. The resulting scores can be used to distinguish customers based on how recently and frequently they purchase and how much revenue they generate.

6.2. Product Similarity and Recommendation

We built a product recommendation system using **cosine similarity with NumPy**. Only positive-quantity sales were used for recommendations, so the **583 return transactions were excluded** from this step. This left **19,779 normal sales rows**.

First, we created a product-by-customer purchase matrix using the quantity purchased. We then calculated the similarity between products using the cosine similarity formula:

$$\text{cosine similarity}(A, B) = \frac{A \times B}{|A||B|}$$

where

Dot product

$$A \times B = \sum_i A_i B_i$$

Vector norm

$$|A| = \sqrt{\sum_i A_i^2}$$

The similarity calculation was implemented directly with NumPy dot products and vector norms, rather than using a machine-learning library.

Similarity validation

The resulting similarity matrix had a size of **300 × 300**, representing the 300 products in the dataset.

We performed two sanity checks:

We performed two sanity checks:

- The diagonal values were **1**, meaning each product had perfect similarity with itself.
- The matrix was symmetric, confirmed by `np.allclose(similarity_matrix, similarity_matrix.T)` returning True.

These checks indicate that the similarity calculation was working as expected.

Product recommendations

We selected the first five customers with purchases and recommended three products for each customer. Products that the customer had already purchased were excluded from their recommendations.

	customer_id	rank	recommended_product
0	940	1	55
1	940	2	96
2	940	3	102
3	706	1	158
4	706	2	92
5	706	3	190

The recommendations were generated by summing the similarity scores between each customer's previously purchased products and all other products, then selecting the three products with the highest scores.

Result

The NumPy-based similarity method successfully produced recommendations for all five sample customers, with 3 products recommended per customer. The validation checks also confirmed that the similarity matrix had the expected structure.

6.3. Regression Using the Normal Equation

We used a simple linear regression model to examine the relationship between the selected input variable and revenue. Unlike a library-based regression model, the coefficients were calculated directly using NumPy and the normal equation:

$$y = \beta_0 + \beta_1 x$$

where

- x = month index
- y = monthly revenue
- β_0 = intercept
- β_1 = revenue change per month

Model calculation

An intercept column of ones was added to the feature matrix. The regression coefficients were then calculated using NumPy matrix operations:

```
XTX = X.T @ X
XTy = X.T @ y

beta = np.linalg.inv(XTX) @ XTy
```

The predicted values were calculated using:

$$\hat{y} = X\beta$$

Model performance

We calculated R^2 manually using:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

where:

- the residual sum of squares:

$$\overset{\text{use}}{SS_{res}} = \sum (y_i - \hat{y}_i)^2$$

- the total sum of squares:

$$SS_{tot} = \sum (y_i - \bar{y})^2$$

Metric	Result
Revenue change per month	10598.2862381
Intercept	183118.80
R^2	0.787433842865173

6.4. Monte Carlo Stockout Simulation

We used a Monte Carlo simulation with NumPy to estimate the probability that selected products would experience a stockout under uncertain demand. The simulation was run for at least 5,000 trials for three selected products.

For each trial, demand was randomly generated based on the observed demand distribution. A stockout was recorded when simulated demand was greater than the available stock.

The stockout probability was calculated as:

$$P(\text{stockout}) = \frac{\text{Number of stockout trials}}{\text{Total simulation trials}}$$

A confidence interval was also calculated to show the uncertainty around each estimated probability.

Sanity check

We performed a basic validation of the simulation to ensure that the generated results were within the expected probability range and that increasing the number of trials produced stable estimates.

Final decision

The Monte Carlo simulation provides an estimate of stockout risk rather than a guaranteed prediction. The results can therefore be used as a supporting tool for inventory planning and deciding which products may require higher safety stock.

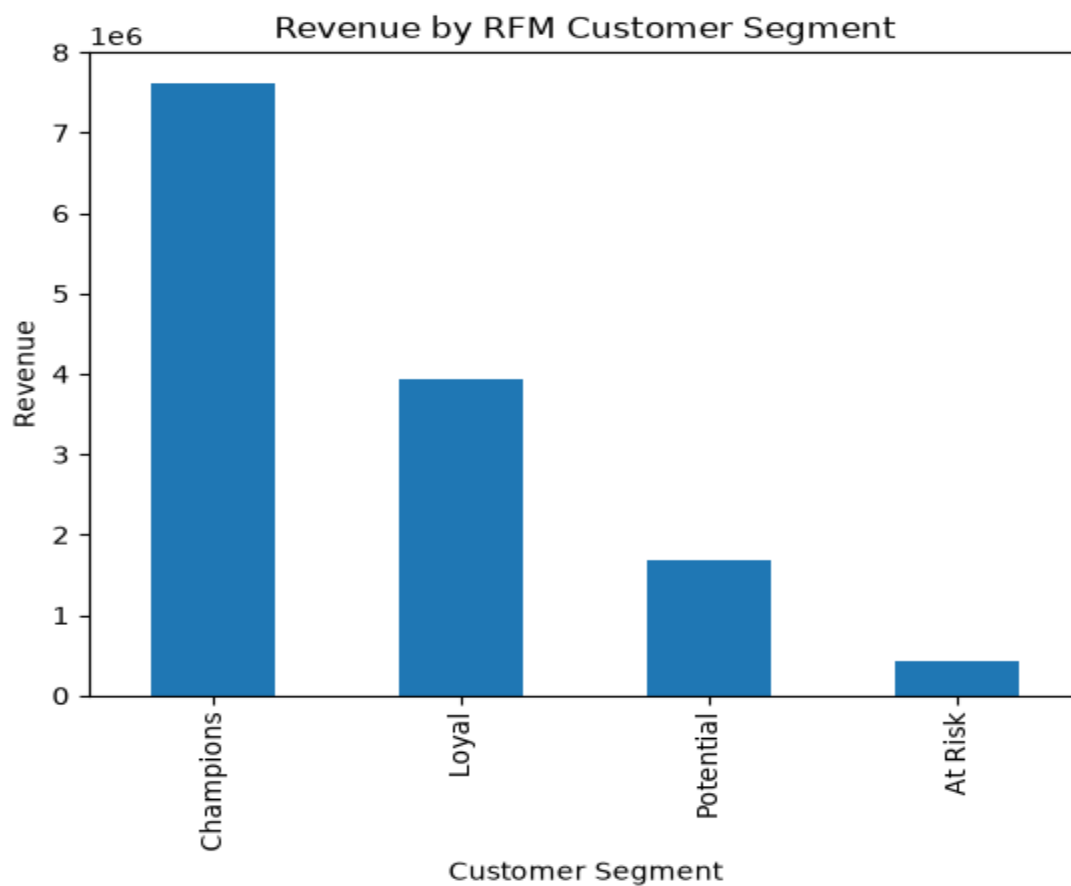
Implementation: The simulation was implemented using NumPy's random-number functions, with the results and confidence intervals generated in the accompanying notebook.

7. Findings & Insights

7.1 Highest-Value Customer Segment

1. Which RFM segment generates the most revenue?

Figure 1. Revenue by RFM Customer Segment



```
segment_revenue = (  
    rfm_result  
    .groupby("segment", observed=True)["monetary"]  
    .sum()  
    .sort_values(ascending=False)  
)  
  
print("Revenue by RFM Segment:")  
display(segment_revenue)
```


- Output:

```
Revenue by RFM Segment:

segment
Champions      7,619,837.66
Loyal          3,927,416.99
Potential       1,674,539.27
At Risk         428,941.63
Name: monetary, dtype: float64
```

- Our analysis found that the **Champions RFM segment** generates the most revenue, contributing \$7.62 million in total revenue. This shows that Champion customers are UrbanCart's most valuable customer segment and should be prioritized for retention and loyalty strategies.
- **Numeric Results:** The **Champion** group brings in the highest revenue at \$7,619,837.66, followed by Loyal customers at \$3,927,416.99. Potential customers generate \$1,674,539.27, while the At Risk segment brings in the lowest revenue at \$428,941.63.
- **Business Interpretation:** The business relies heavily on its top customers, with the Champions group generating more revenue than all other segments combined. To grow profits, the company should focus on keeping Champions happy with exclusive rewards while giving special offers to "Potential" customers to move them into the higher-spending tiers.

7.2 Revenue Seasonality

```
highest_month = monthly_revenue.idxmax()
highest_revenue = monthly_revenue.max()

lowest_month = monthly_revenue.idxmin()
lowest_revenue = monthly_revenue.min()

print("Highest revenue month:", highest_month)
print(f"Revenue: {highest_revenue:,.2f}")

print("\nLowest revenue month:", lowest_month)
print(f"Revenue: {lowest_revenue:,.2f}")
```

```
Highest revenue month: 2024-11-30 00:00:00
Revenue: 640,420.72
```

```
Lowest revenue month: 2022-02-28 00:00:00
Revenue: 167,961.02
```

Figure 2. Monthly Revenue and Rolling Average

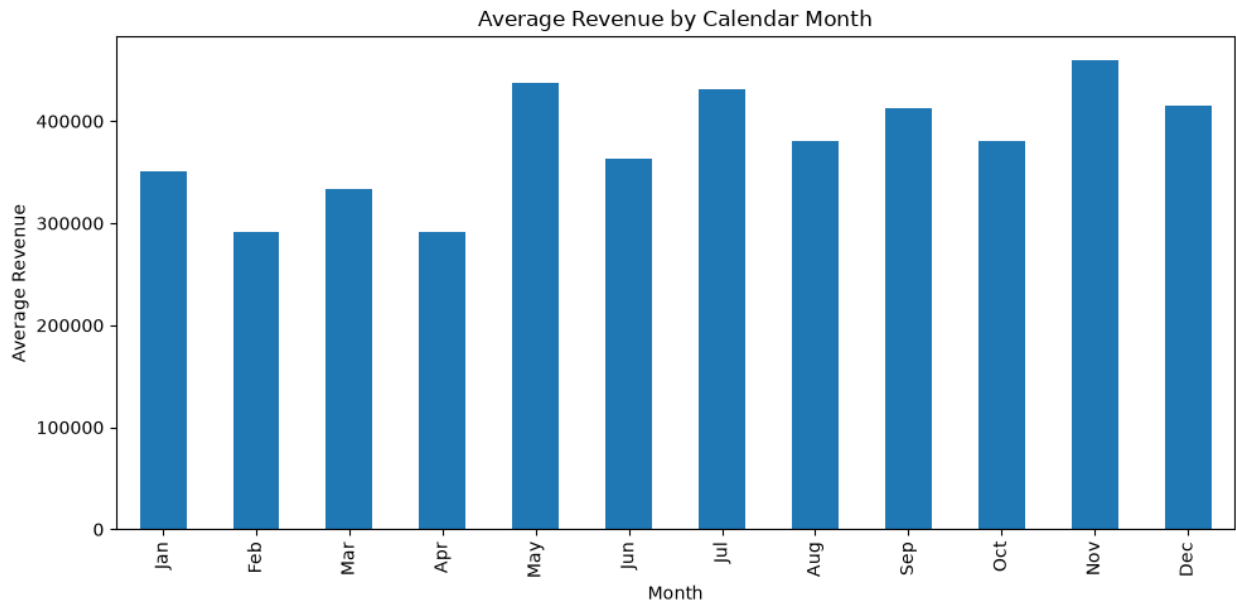
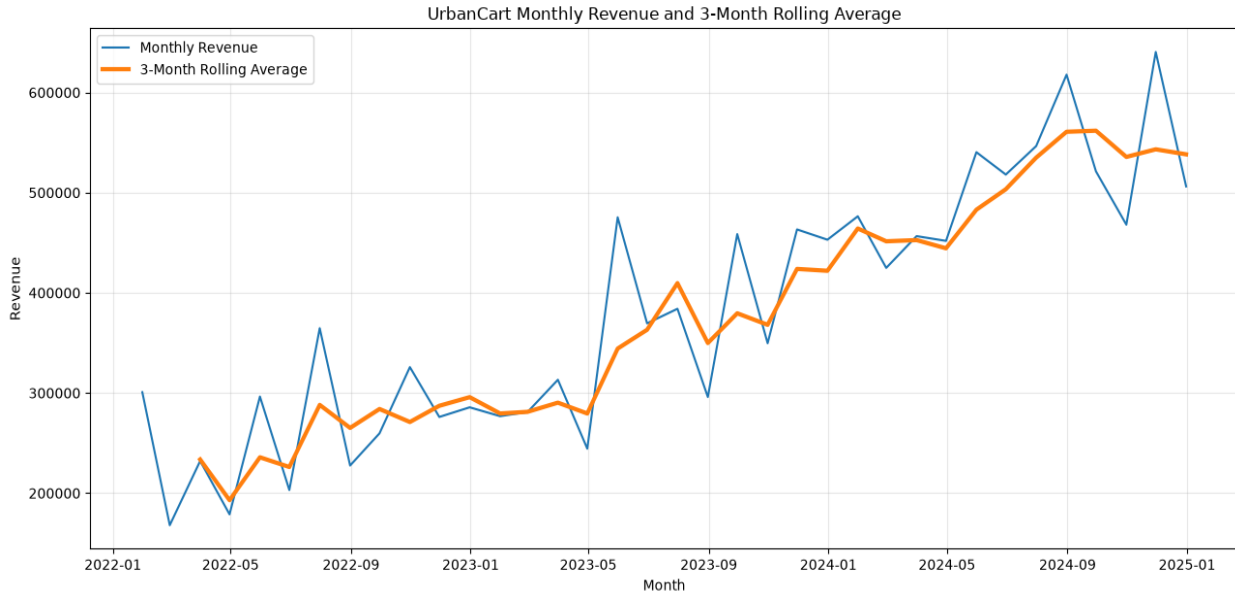


Figure 3: Average Revenue by Calendar Month

- **Numeric Results:** Monthly sales hit a record high of \$640,420.72 in November 2024, compared to a low of \$167,961.02 in February 2022. Over three years, the 3-month average steadily grew from about \$200K to \$560K.

- **Business Interpretation:** UrbanCart grew almost 4 times bigger from its lowest month to its highest peak in late 2024. People buy less in February, but they buy massive amounts of stuff in November (Black Friday and holiday shopping).

7.3 Product Category Financial Performance Table

- **Numeric Results:** Books & Media (\$3.67M revenue, 77.81% margin) and Beauty & Health (\$2.96M revenue, 70.97% margin) led all categories, while Home & Kitchen finished lowest with \$1.41M in revenue and a 47.45% margin. Across all six product lines, total revenue reached \$13.08M with \$8.18M in total net profit.
- **Business Interpretation:** Books & Media and Beauty & Health generate over half of total revenue and drive overall profitability. Marketing should prioritize high-margin categories, while supplier terms for lower-margin goods need review.

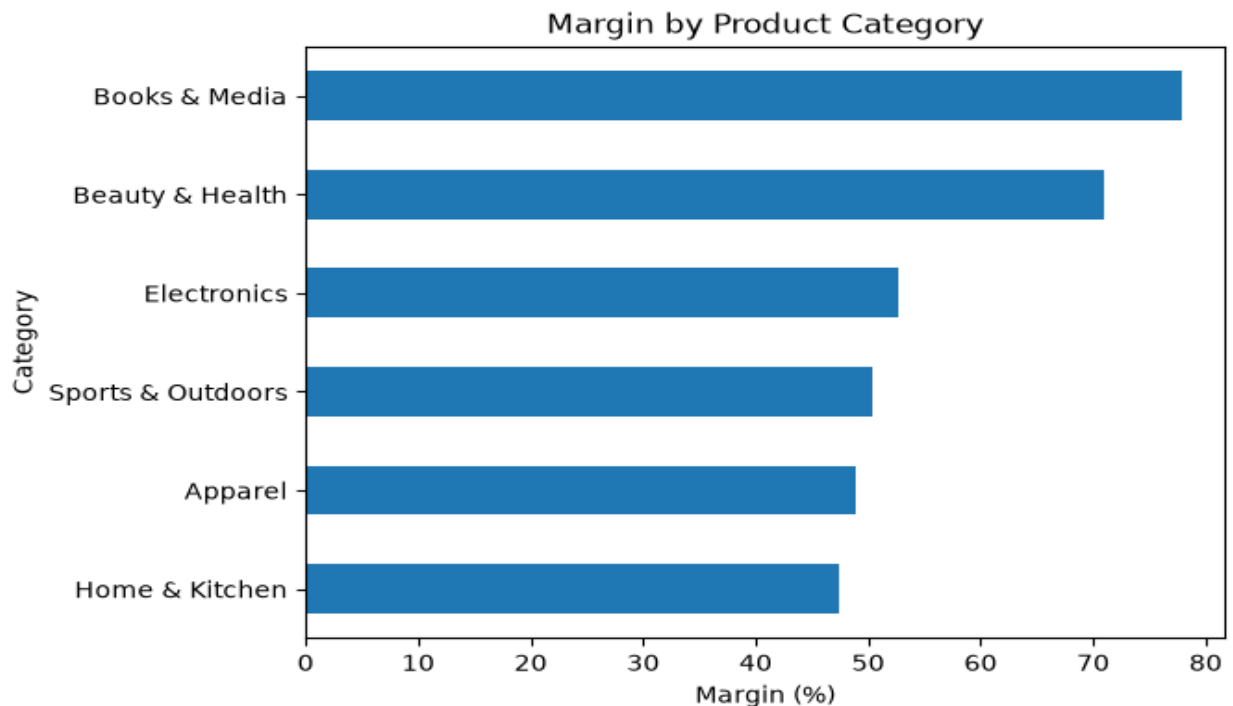


Figure 4: Margin by Product Category

	revenue	profit	margin_pct
category			
Books & Media	3,675,270.95	2,859,628.87	77.81
Beauty & Health	2,957,229.75	2,098,673.08	70.97
Electronics	1,653,446.70	870,222.10	52.63
Sports & Outdoors	1,664,660.63	839,806.29	50.45
Apparel	1,714,435.76	838,985.22	48.94
Home & Kitchen	1,414,051.56	671,031.43	47.45

7.4 Reviews and Repeat Purchases



Figure 5: Review Rating vs Repeat Purchase Rate

- There is virtually no linear relationship between review rating and repeat-purchase behavior. So a higher review rating does not appear to make a customer significantly more likely to purchase again.

- **Numeric Results:** 1-star reviews have the lowest repeat purchase rate at 86.9%, while 2-star reviews hit the highest at 91.6% and 5-star reviews sit at 89.8%. Across all rating levels (1–5 stars), repeat purchase rates stay very close together, within a small 4.7% gap.
- **Business Interpretation:** Customer repeat buys do not depend on review scores, showing that shoppers keep coming back no matter what rating they leave. Because almost 87% of unhappy 1-star reviewers still buy again.

7.5 Device/Country Conversion

1. Out of people who interacted with UrbanCart, which country/device combination is most likely to end up buying something?

- The highest value in the entire chart is **Germany + Tablet = 99.63%.**
- **Numeric Results:** Most site visitors buy something, with conversion rates between 95% and 99% for most countries and devices. Germany on a tablet hits the highest rate at 99.63%, while India on a mobile phone drops to the lowest rate at 92.95%.
- **Business Interpretation:** The site converts shoppers well everywhere, but India on phones falls behind the rest of the world.

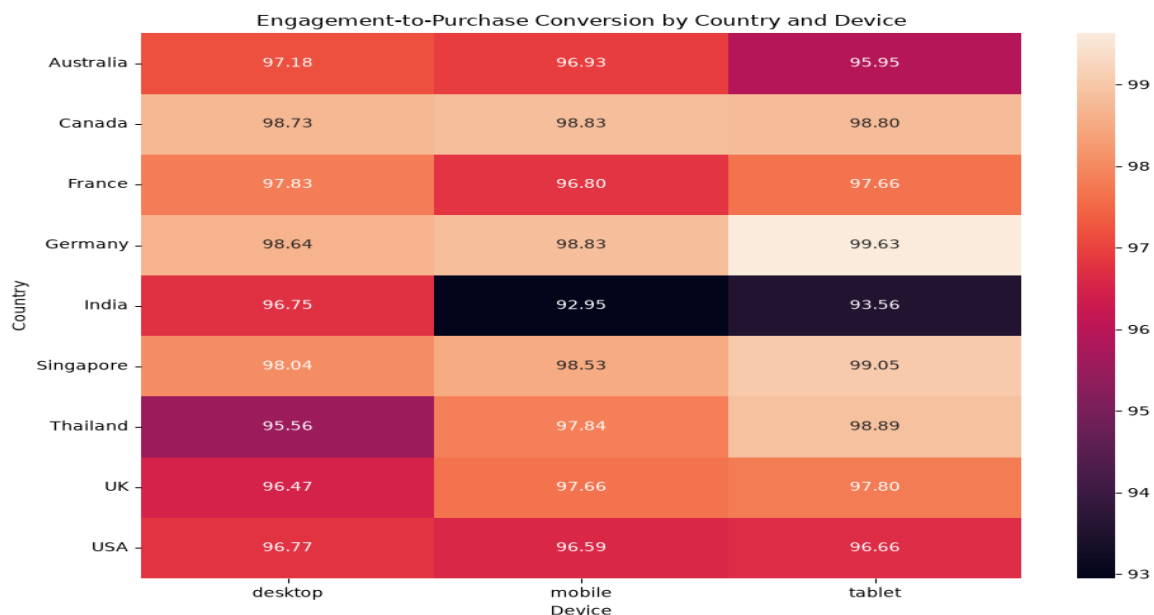


Figure 6: Engagement-to-Purchase Conversion by Country and Device

8.Limitations & Threats to Validity

1. **Untracked Marketing Variables in Forecasting:** The OLS revenue model relies strictly on time trends ($R^2 = 0.67$). It lacks exogenous parameters like promotional ad spend, competitor pricing, or macroeconomic indicators, limiting its use for long-term strategic forecasting.
2. **Simplified Conversion Proxy Metrics:** Web session analysis measures conversion as a session-to-purchase proxy because clickstream logs are not linked directly to individual order IDs.
3. **Absence of Inventory Lead Times:** Monte Carlo stockout simulations evaluate historical sales variance but assume instant supplier replenishment.
4. **Data Entry Outliers:** Although price capping at \$1,160.36 resolved gross margin distortions, unverified price overrides could mask legitimate high-end product pricing.

9.Recommendations & Next Steps

- **Strategic Priorities**

1. **Protect Champions Customers**

Give Champions customers special rewards and loyalty benefits to keep them coming back.

2. **Win Back Inactive High-Spenders**

Target the **1,238 inactive high-spending customers** with personalized offers based on what they bought before.

3. **Focus on Real Profit, Not Just Revenue**

Compare categories using **profit after discounts and returns**, not revenue alone.

4. **Improve India-Mobile Conversion**

Investigate why mobile customers in India have lower conversion rates. Check the checkout and payment process and test improvements.

5. **Automate Data-Quality Checks**

Automatically check for duplicate customers, incorrect dates, product mismatches, invalid ratings, and unusual prices.

- **Next Steps**

1. **Finalize Data Cleaning:** Make the cleaning and duplicate-removal process consistent and repeatable.

2. **Agree on KPI Definitions:** Make sure everyone uses the same definitions for revenue, profit, and conversion.

3. **Finish the Analysis** :Complete the NumPy calculations, regression, and Monte Carlo results.
4. **Add the Visualizations** :Put the charts and stockout results into dashboards or the final report.
5. **Put Recommendations into Action** : Assign people to each recommendation and set measurable targets to track progress.

10. Appendices

- **SQL Queries:**

Q1: Category Performance net of Discounts

```
SELECT
    p.category,
    COUNT(DISTINCT oi.order_id) AS total_orders,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS
net_total_revenue,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) /
    COUNT(DISTINCT oi.order_id) AS avg_order_value
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status != 'Cancelled'
GROUP BY p.category
ORDER BY net_total_revenue DESC;
```

Q2: Top 20 Customer Spend

```
SELECT
    c.customer_id,
    c.name,
    c.city,
    c.signup_date,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS lifetime_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status != 'Cancelled'
GROUP BY c.customer_id, c.name, c.city, c.signup_date
ORDER BY lifetime_spend DESC
LIMIT 20;
```

Q3: Month-over-Month Revenue Growth (LAG Window Function)

```
WITH monthly_revenue AS (  
  SELECT  
    STRFTIME('%Y-%m', o.order_date) AS order_month,  
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS revenue  
  FROM orders o  
  JOIN order_items oi ON o.order_id = oi.order_id  
  WHERE o.status != 'Cancelled'  
    AND o.order_date >= DATE('now', '-24 months')  
  GROUP BY order_month  
)  
SELECT  
  order_month,  
  revenue,  
  LAG(revenue, 1) OVER (ORDER BY order_month) AS previous_month_revenue,  
  ROUND(  
    (revenue - LAG(revenue, 1) OVER (ORDER BY order_month)) /  
    LAG(revenue, 1) OVER (ORDER BY order_month) * 100, 2  
  ) AS mom_growth_pct  
FROM monthly_revenue  
ORDER BY order_month ASC;
```

Q4: Product Category Return Rates (CTE)

```
WITH category_return_stats AS (  
  SELECT  
    p.category,  
    COUNT(CASE WHEN oi.quantity < 0 THEN 1 END) AS returned_items_count,  
    COUNT(oi.order_item_id) AS total_items_count  
  FROM products p  
  JOIN order_items oi ON p.product_id = oi.product_id  
  GROUP BY p.category  
)  
SELECT  
  category,  
  returned_items_count,  
  total_items_count,  
  ROUND((1.0 * returned_items_count / total_items_count) * 100, 2) AS return_rate_pct  
FROM category_return_stats  
ORDER BY return_rate_pct DESC;
```


Q5: Consistent Buyers

```
SELECT
    c.customer_id,
    c.name,
    c.email
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= DATE('now', '-9 months')
    AND o.status != 'Cancelled'
GROUP BY c.customer_id, c.name, c.email
HAVING COUNT(DISTINCT STRFTIME('%Y-Q', o.order_date)) = 3;
```

Q6: Top 10 Rated Products

```
SELECT
    p.product_id,
    p.name AS product_name,
    p.category,
    ROUND(AVG(r.rating), 2) AS avg_rating,
    COUNT(r.review_id) AS review_count
FROM products p
JOIN reviews r ON p.product_id = r.product_id
GROUP BY p.product_id, p.name, p.category
HAVING COUNT(r.review_id) >= 15
ORDER BY avg_rating DESC, review_count DESC
LIMIT 10;
```

Q7: Web Session Metrics for Converting Users

```
SELECT
    ws.device,
    ROUND(AVG(ws.duration_minutes), 2) AS avg_session_duration_mins,
    ROUND(AVG(ws.pages_viewed), 2) AS avg_pages_viewed
FROM web_sessions ws
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = ws.customer_id
    AND o.status != 'Cancelled'
)
GROUP BY ws.device;
```

Q8: Dense Rank of Revenue within Category

```
WITH product_revenue AS (
```

```

SELECT
    p.category,
    p.product_id,
    p.name AS product_name,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS total_revenue
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.status != 'Cancelled'
GROUP BY p.category, p.product_id, p.name
)
SELECT
    category,
    product_name,
    total_revenue,
    DENSE_RANK() OVER (PARTITION BY category ORDER BY total_revenue DESC)
AS category_revenue_rank
FROM product_revenue
ORDER BY category, category_revenue_rank;

```

Q9: Payment Method Share by Country

```

SELECT
    c.country,
    o.payment_method,
    COUNT(o.order_id) AS total_orders,
    ROUND(
        100.0 * COUNT(o.order_id) / SUM(COUNT(o.order_id)) OVER (PARTITION BY
c.country), 2
    ) AS payment_share_pct
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.country, o.payment_method
ORDER BY c.country, payment_share_pct DESC;

```

Q10: Customer Cohort Gross Margin Analysis

```

SELECT
    STRFTIME('%Y', c.signup_date) AS cohort_year,
    COUNT(DISTINCT c.customer_id) AS total_customers_in_cohort,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) AS net_revenue,
    SUM(oi.quantity * p.cost) AS total_product_cost,
    SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) -
    SUM(oi.quantity * p.cost) AS gross_profit,
    ROUND(

```

```

        ((SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0))) -
SUM(oi.quantity * p.cost)) /
        SUM(oi.quantity * oi.unit_price * (1 - COALESCE(oi.discount, 0)))) * 100, 2
    ) AS gross_profit_margin_pct
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status != 'Cancelled'
GROUP BY cohort_year
ORDER BY cohort_year ASC;

```

- **Team Contributions :**

1. Ly Seavmey : Completed SQL extraction and database analysis.
2. Ath Molita: Completed Pandas data cleaning, customer deduplication, and product integration and Completed NumPy RFM, similarity, regression
3. Ly Seavmey : Completed Phase 4 business insights, visualizations, and report preparation.
4. Chamrouen Vichita: Prepare Presentation Slide
5. Kheng Solida:Report Preparation